

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: KIỂM THỬ PHẦN MỀM

KIỂM THỬ HỆ THỐNG WEBSITE CỦA HÀNG TIỆN
LỢI VỚI CÔNG CỤ MANTIS BUG TRACKER

Sinh viên thực hiện : **Đinh Minh Đức**
Nguyễn Quang Huy
Nguyễn Công Thành
Nguyễn Hoàng Anh

Ngành : **Công nghệ thông tin**
Giảng viên hướng dẫn : **ThS. Lê Thị Thùy Trang**

BÁO CÁO LÀM VIỆC NHÓM

Tên nhóm: Nhóm Kiểm thử Phần mềm DNU - 7coMART

Nhóm trưởng: Đinh Minh Đức

Phân công nhiệm vụ và nhận xét mức độ hoàn thành từng thành viên:

Nhóm trưởng Đinh Minh Đức chịu trách nhiệm chung về thiết kế và cấu trúc luồng của báo cáo, thiết lập môi trường và cấu hình bìa. Thành viên Nguyễn Quang Huy thiết kế và xây dựng chi tiết bộ kịch bản kịch bản kiểm thử (Test Cases). Thành viên Nguyễn Công Thành chịu trách nhiệm thực thi kiểm thử và thu thập kết quả thực tế trên giao diện. Thành viên Nguyễn Hoàng Anh chịu trách nhiệm nạp lỗi lên Mantis và tổng hợp các độ đo chất lượng phần mềm nâng cao. Toàn bộ các thành viên đều hoàn thành xuất sắc nhiệm vụ được giao, đóng góp tích cực và hiệu quả vào sự thành công của bài tập lớn.

Kết quả làm việc nhóm (theo tỷ lệ %)

STT	MSV	Họ và tên	Lớp	Mức độ đóng góp kết quả làm việc nhóm (%)
1	1041060211	Đinh Minh Đức	CNTT15-02	25% (Trưởng nhóm - Thiết kế)
2	1041060212	Nguyễn Quang Huy	CNTT15-02	25% (Thành viên - Viết Test case)
3	1041060213	Nguyễn Công Thành	CNTT15-02	25% (Thành viên - Thực thi test)
4	1041060214	Nguyễn Hoàng Anh	CNTT15-02	25% (Thành viên - Log Mantis)

NHẬN XÉT
(Của giảng viên chấm)

.....

.....

.....

.....

.....

.....

STT	MSV	Họ và tên	Lớp	Điểm	
				Điểm số	Điểm chữ
1	1041060211	Đinh Minh Đức	CNTT15-02		
2	1041060212	Nguyễn Quang Huy	CNTT15-02		
3	1041060213	Nguyễn Công Thành	CNTT15-02		
4	1041060214	Nguyễn Hoàng Anh	CNTT15-02		

Ngày ... tháng ... năm 2026

CÁN BỘ CHẤM THI 1
(Ký ghi rõ họ tên)

CÁN BỘ CHẤM THI 2
(Ký ghi rõ họ tên)

Mục lục

1	TỔNG QUAN VỀ DỰ ÁN VÀ CƠ SỞ LÝ THUYẾT	1
1.1	Giới thiệu dự án Website Cửa hàng Tiện lợi 7coMART	1
1.1.1	Bối cảnh dự án	1
1.1.2	Kiến trúc kỹ thuật của hệ thống	1
1.1.3	Các mô-đun chức năng cốt lõi	2
1.2	Tổng quan về Kiểm thử phần mềm	3
1.2.1	Mục tiêu và Tầm quan trọng	3
1.2.2	Áp dụng 7 nguyên tắc kiểm thử phần mềm vào dự án 7coMART	3
1.2.3	Phân biệt Kiểm thử Chức năng và Kiểm thử Phi chức năng	4
1.3	Tổng quan về công cụ quản lý lỗi Mantis Bug Tracker	4
2	LẬP KẾ HOẠCH VÀ THIẾT KẾ KỊCH BẢN KIỂM THỬ	5
2.1	Phạm vi kiểm thử theo từng Sprint	5
2.2	Áp dụng các kỹ thuật kiểm thử Hộp đen (Black-box Testing)	5
2.2.1	Kỹ thuật Phân lớp tương đương (Equivalence Partitioning)	6
2.2.2	Kỹ thuật Phân tích giá trị biên (Boundary Value Analysis)	6
2.2.3	Kỹ thuật Bảng quyết định (Decision Table)	7
2.3	Danh sách Kịch bản kiểm thử (Test Cases) chi tiết	7
3	THỰC THI KIỂM THỬ VÀ QUẢN LÝ LỖI TRÊN MANTIS	10
3.1	Thiết lập môi trường và Thực thi kiểm thử	10
3.1.1	Môi trường thực thi kiểm thử	10
3.1.2	Quy trình và Quy ước ghi nhận kết quả thực thi	10
3.2	Quản lý vòng đời lỗi chuyên nghiệp với Mantis Bug Tracker	11
3.2.1	Sơ đồ vòng đời lỗi trong LaTeX bằng TikZ	11
3.2.2	Quy trình viết Báo cáo lỗi (Defect Report) chuẩn chỉnh	11
3.3	Thống kê và Chi tiết một số lỗi (Bugs) tiêu biểu	12
3.3.1	Bug 1: Lỗi hỏng bảo mật SQL Injection tại Form Đăng nhập	12

3.3.2	Bug 2: Lỗi 500 Internal Server Error khi tìm kiếm từ khóa tiếng Việt chứa Unicode	13
3.3.3	Bug 3: Tấn công thay đổi gói tin sửa đổi số lượng sản phẩm âm trong Giỏ hàng	14
4	KIỂM THỬ HỒI QUY VÀ ĐÁNH GIÁ CHẤT LƯỢNG	16
4.1	Thực hiện Kiểm thử hồi quy (Regression Testing)	16
4.1.1	Định nghĩa và Tầm quan trọng	16
4.1.2	Quy trình và lỗi của Lập trình viên và kiểm thử lại của Tester	16
4.2	Phân tích các độ đo chất lượng phần mềm nâng cao	17
4.2.1	Độ đo Mật độ lỗi (Defect Density - DD)	17
4.2.2	Độ đo Hiệu quả loại bỏ lỗi (Defect Removal Efficiency - DRE)	17
4.3	Báo cáo kiểm thử tổng hợp (Test Summary Report)	18
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
5.1	Kết quả đạt được của đề tài	20
5.2	Bài học kinh nghiệm rút ra	20
5.3	Hướng phát triển trong tương lai - Kiểm thử tự động (Automation Testing) . . .	21
5.3.1	Mã nguồn mẫu kịch bản kiểm thử tự động bằng Python và Playwright . .	21
A	Phụ lục: Hướng dẫn cài đặt và Cấu hình hệ thống Kiểm thử	23

Danh sách bảng

1.1	So sánh Kiểm thử Chức năng và Phi chức năng tại 7coMART	4
2.1	Bảng quyết định logic áp dụng mã giảm giá tại 7coMART	7
2.2	Kịch bản kiểm thử chi tiết - Sprint 1 (Xác thực & Tìm kiếm)	8
2.3	Kịch bản kiểm thử chi tiết - Sprint 2 (Giỏ hàng & Đặt hàng)	9
4.1	Bảng tổng kết thực thi kiểm thử theo từng mô-đun chức năng	18
4.2	Thống kê số lượng lỗi phát hiện phân loại theo Mức độ nghiêm trọng	18

Danh sách hình vẽ

1.1	Giao diện trang chủ mua sắm trực tuyến của hệ thống cửa hàng tiện lợi 7coMART	2
3.1	Giao diện bảng điều khiển quản lý và theo dõi lỗi Mantis Bug Tracker của dự án 7coMART	12
3.2	Minh chứng lỗi SQL Injection đăng nhập thành công vào quyền Admin và ticket tương ứng trên Mantis	13
3.3	Lỗi 500 Internal Server Error hiển thị trên giao diện và ghi nhận Console khi tìm kiếm tiếng Việt có dấu	14
3.4	Tổng quan giỏ hàng bị can thiệp giá trị số lượng âm khiến hóa đơn thanh toán mang giá trị âm	15
4.1	Biểu đồ thống kê tỷ lệ phần trăm phân bố trạng thái lỗi và mức độ nghiêm trọng trích xuất từ hệ thống Mantis Bug Tracker	19

Chương 1

TỔNG QUAN VỀ DỰ ÁN VÀ CƠ SỞ LÝ THUYẾT

1.1 Giới thiệu dự án Website Cửa hàng Tiện lợi 7coMART

1.1.1 Bối cảnh dự án

Trong kỷ nguyên số hóa và sự bùng nổ của mô hình thương mại điện tử giao hàng tức thì (Quick Commerce), việc vận hành một hệ thống cửa hàng tiện lợi trực tuyến đòi hỏi tính ổn định và tốc độ xử lý giao dịch thời gian thực cao. Dự án website cửa hàng tiện lợi **7coMART** được thiết kế nhằm đáp ứng nhu cầu mua sắm nhu yếu phẩm nhanh chóng của khách hàng trực tuyến, tích hợp các tính năng đặt hàng, quản lý danh mục và phân bổ kho hàng tự động.

1.1.2 Kiến trúc kỹ thuật của hệ thống

Hệ thống 7coMART được xây dựng dựa trên những công nghệ hiện đại, đảm bảo hiệu năng và khả năng mở rộng:

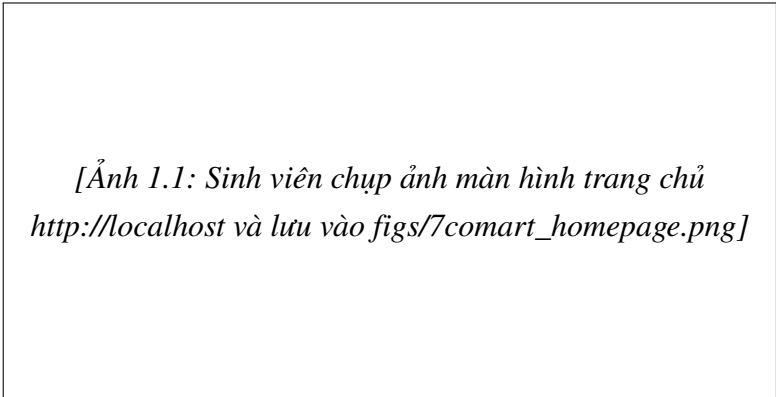
- **Frontend (Giao diện người dùng):** ReactJS kết hợp với TypeScript giúp xây dựng giao diện Single Page Application (SPA) mượt mà, phản hồi nhanh và tối ưu hóa trải nghiệm người dùng trên cả thiết bị di động lẫn máy tính để bàn.
- **Backend (Xử lý nghiệp vụ):** Python FastAPI cung cấp hệ thống API RESTful hiệu năng cực cao nhờ cơ chế bất đồng bộ (async-first), tự động sinh tài liệu kiểm thử tương tác Swagger UI thuận tiện cho việc phát hiện lỗi.
- **Database (Lưu trữ dữ liệu):** PostgreSQL 16 đóng vai trò cơ sở dữ liệu quan hệ chính, đảm bảo tính toàn vẹn dữ liệu giao dịch nhờ cơ chế ACID nghiêm ngặt.

- **Caching & Locking:** Redis 7 được tích hợp để lưu trữ cache giỏ hàng tạm thời và thực hiện cơ chế khóa bi quan (Pessimistic Locking) ngăn chặn hiện tượng bán quá mức (over-selling) khi có nhiều khách hàng mua cùng một mặt hàng tại một thời điểm.
- **Triển khai:** Đóng gói toàn bộ hệ thống bằng Docker Compose giúp đồng bộ môi trường phát triển và kiểm thử.

1.1.3 Các mô-đun chức năng cốt lõi

Hệ thống tập trung vào 4 luồng nghiệp vụ lớn của khách hàng:

1. **Mô-đun Xác thực (Authentication):** Cho phép người dùng đăng ký tài khoản, đăng nhập hệ thống, quản lý thông tin cá nhân. Phân quyền truy cập rõ ràng dựa trên JWT Token giữa các vai trò: Khách hàng (Customer), Nhân viên quản lý đơn hàng (Staff) và Quản trị viên (Admin).
2. **Mô-đun Danh mục và Sản phẩm (Catalog Search):** Hiển thị danh sách sản phẩm trực quan, hỗ trợ tìm kiếm sản phẩm theo tên và lọc theo các danh mục như thực phẩm, đồ uống, hóa mỹ phẩm.
3. **Mô-đun Giỏ hàng (Cart):** Hỗ trợ thêm sản phẩm, cập nhật số lượng trực tiếp trong giỏ hàng, đồng bộ hóa thời gian thực và tính toán giá trị tạm tính của đơn hàng.
4. **Mô-đun Đặt hàng và Xử lý kho (Order Processing):** Thực hiện lưu vết đơn hàng, phân bổ kho hàng tự động áp dụng thuật toán FEFO (First Expired First Out - Lô hàng hết hạn trước xuất trước) nhằm tối ưu hóa việc quản lý hạn sử dụng của sản phẩm tiện lợi.



[Ảnh 1.1: Sinh viên chụp ảnh màn hình trang chủ <http://localhost> và lưu vào `figs/7comart_homepage.png`]

Hình 1.1: Giao diện trang chủ mua sắm trực tuyến của hệ thống cửa hàng tiện lợi 7coMART

Nhận xét giao diện: Như được minh họa tại Hình 1.1, giao diện trang chủ của 7coMART được thiết kế hiện đại, responsive hoàn toàn. Các khối danh mục sản phẩm tiện lợi được bố trí trực quan giúp người dùng dễ dàng tìm kiếm và lựa chọn sản phẩm nhanh chóng. Ô tìm kiếm nổi bật ở thanh Header là tiêu điểm cho các kịch bản kiểm thử tìm kiếm tiếp theo.

1.2 Tổng quan về Kiểm thử phần mềm

1.2.1 Mục tiêu và Tầm quan trọng

Kiểm thử phần mềm là quá trình khảo sát thực nghiệm một sản phẩm phần mềm nhằm cung cấp cho các bên liên quan thông tin về chất lượng của sản phẩm đó. Đối với 7coMART, kiểm thử phần mềm giúp:

- Phát hiện sớm các lỗi logic nghiệp vụ trong việc tính tiền giỏ hàng, trừ kho hàng FEFO trước khi hệ thống chính thức vận hành thương mại.
- Đảm bảo an toàn bảo mật, chống các cuộc tấn công đánh cắp tài khoản hoặc can thiệp tham số giá tiền.
- Khẳng định hệ thống chạy đúng yêu cầu đặc tả chức năng và đem lại trải nghiệm mua sắm ổn định nhất cho người dùng.

1.2.2 Áp dụng 7 nguyên tắc kiểm thử phần mềm vào dự án 7coMART

1. **Kiểm thử chỉ ra sự hiện diện của lỗi (Testing shows presence of defects):** Các kịch bản kiểm thử được thiết kế nhằm tìm kiếm và chứng minh website 7coMART vẫn tồn tại lỗi (như lỗi SQLi, lỗi số lượng âm). Kiểm thử không thể chứng minh hệ thống hoàn hảo 100% không có lỗi.
2. **Kiểm thử toàn diện là bất khả thi (Exhaustive testing is impossible):** Số lượng sản phẩm kết hợp trong giỏ hàng và dữ liệu nhập vào form là vô hạn. Nhóm tập trung thiết kế các kịch bản kiểm thử trọng tâm cho luồng mua sắm chính bằng cách phân lớp dữ liệu.
3. **Kiểm thử càng sớm càng tốt (Early testing):** Việc lập kế hoạch và thiết kế kịch bản test được nhóm triển khai ngay sau khi chốt tài liệu đặc tả yêu cầu chức năng, giúp phát hiện lỗi logic ngay trên bản thiết kế trước khi lập trình viên bắt tay viết code.
4. **Sự tập trung của lỗi (Defects clustering):** Lỗi thường tập trung nhiều ở những mô-đun phức tạp. Trong 7coMART, lỗi chủ yếu nằm ở mô-đun đặt hàng kết hợp với khóa đồng thời trừ kho PostgreSQL và kiểm soát hạn sử dụng.
5. **Nghịch lý thuốc trừ sâu (Pesticide paradox):** Nếu liên tục lặp lại một bộ test cases cũ, nhóm sẽ không tìm thêm được lỗi mới. Vì vậy, bộ kịch bản test luôn được cập nhật, bổ sung các ca kiểm thử biên và giá trị ngẫu nhiên.
6. **Kiểm thử phụ thuộc vào ngữ cảnh (Testing is context dependent):** 7coMART là một ứng dụng thương mại điện tử, do đó ngữ cảnh kiểm thử cực kỳ coi trọng tính toàn vẹn dữ liệu giỏ hàng, giao dịch đặt hàng và tính bảo mật của mã hóa mật khẩu.

7. **Sự ngộ nhận về việc không có lỗi (Absence-of-errors fallacy):** Một hệ thống 7coMART được test sạch bóng lỗi kỹ thuật vẫn là thất bại nếu giao diện quá khó dùng, luồng đặt hàng rườm rà khiến khách hàng bỏ đi. Nhóm luôn kiểm định dựa trên trải nghiệm khách hàng thực tế.

1.2.3 Phân biệt Kiểm thử Chức năng và Kiểm thử Phi chức năng

Bảng 1.1: So sánh Kiểm thử Chức năng và Phi chức năng tại 7coMART

Tiêu chí	Kiểm thử Chức năng	Kiểm thử Phi chức năng
Mục tiêu	Kiểm tra hệ thống làm ĐÚNG chức năng yêu cầu hay không.	Kiểm tra hệ thống hoạt động TỐT như thế nào (hiệu năng, bảo mật).
Đối tượng	Đăng nhập thành công, tìm thấy sản phẩm, thêm đúng số lượng giỏ hàng, tạo đơn hàng trừ kho đúng lô FEFO.	Thời gian phản hồi API dưới 200ms, mã hóa mật khẩu bằng bcrypt, chống tấn công SQLi, XSS.
Công cụ	Manual test trên trình duyệt, Postman gọi API.	JMeter (tải), OWASP ZAP (bảo mật), Lighthouse.

1.3 Tổng quan về công cụ quản lý lỗi Mantis Bug Tracker

Mantis Bug Tracker (MantisBT) là một hệ thống theo dõi lỗi phần mềm mã nguồn mở dựa trên web. Công cụ cung cấp một quy trình chuyên nghiệp giúp Tester ghi nhận lỗi một cách chi tiết và phân công cho Lập trình viên xử lý, theo sát vòng đời lỗi từ lúc phát hiện cho tới khi được khắc phục hoàn toàn. Sự tích hợp Mantis giúp tăng cường tính minh bạch, rút ngắn khoảng cách giao tiếp giữa các thành viên dự án và tự động hóa các đo lường báo cáo chất lượng phần mềm.

Chương 2

LẬP KẾ HOẠCH VÀ THIẾT KẾ KỊCH BẢN KIỂM THỬ

2.1 Phạm vi kiểm thử theo từng Sprint

Nhằm đảm bảo quá trình kiểm thử được triển khai khoa học, nhóm đã phân chia phạm vi kiểm thử thành 02 Sprint thực hành trọng tâm, tập trung vào luồng tương tác cốt lõi của khách hàng:

- **Sprint 1 (Xác thực và Tra cứu sản phẩm):**
 - Tính năng Đăng ký tài khoản mới và Đăng nhập hệ thống (Phân quyền Customer/Staff/Admin).
 - Tính năng Tìm kiếm sản phẩm theo tên và Lọc sản phẩm theo danh mục ngành hàng.
- **Sprint 2 (Giỏ hàng và Quy trình đặt hàng):**
 - Các thao tác thêm sản phẩm vào giỏ, điều chỉnh tăng/giảm số lượng sản phẩm, xóa sản phẩm khỏi giỏ hàng.
 - Luồng xác nhận đơn hàng, ràng buộc số lượng tồn kho thực tế, áp dụng quy tắc FEFO khi xuất kho hàng hóa.

2.2 Áp dụng các kỹ thuật kiểm thử Hộp đen (Black-box Testing)

Kiểm thử hộp đen tập trung hoàn toàn vào đầu vào và đầu ra của phần mềm dựa trên tài liệu đặc tả chức năng, không đòi hỏi Tester phải hiểu rõ cấu trúc mã nguồn bên trong Python

FastAPI hay React. Nhóm áp dụng 3 kỹ thuật kinh điển để tối ưu hóa bộ kịch bản kiểm thử:

2.2.1 Kỹ thuật Phân lớp tương đương (Equivalence Partitioning)

Kỹ thuật này chia miền đầu vào của chương trình thành các lớp dữ liệu tương đương nhau, đại diện cho hành vi xử lý giống nhau của hệ thống. Ta chỉ cần chọn một giá trị đại diện trong mỗi lớp để kiểm thử, giúp giảm thiểu đáng kể số ca kiểm thử cần chạy.

Áp dụng cho Form Đăng nhập 7coMART:

- **Trường Email:**

- Lớp hợp lệ (EC_1): Địa chỉ email đúng cấu trúc tiêu chuẩn (Ví dụ: khach1@gmail.com).
- Lớp không hợp lệ (EC_2): Email thiếu ký tự @ (Ví dụ: khach1gmail.com).
- Lớp không hợp lệ (EC_3): Email thiếu tên miền (Ví dụ: khach1@.com hoặc khach1@gmail).
- Lớp không hợp lệ (EC_4): Email bỏ trống.

- **Trường Mật khẩu:**

- Lớp hợp lệ (EC_5): Mật khẩu có độ dài từ 8 đến 20 ký tự (Ví dụ: password123).
- Lớp không hợp lệ (EC_6): Mật khẩu quá ngắn dưới 8 ký tự (Ví dụ: pass12).
- Lớp không hợp lệ (EC_7): Mật khẩu quá dài trên 20 ký tự.
- Lớp không hợp lệ (EC_8): Mật khẩu để trống.

2.2.2 Kỹ thuật Phân tích giá trị biên (Boundary Value Analysis)

Lỗi phần mềm thường tập trung rất cao ở ranh giới của các miền giá trị đầu vào thay vì ở giữa. Kỹ thuật này lựa chọn các giá trị tại biên và các giá trị ngay sát biên để thực thi kiểm thử.

Áp dụng cho số lượng sản phẩm thêm vào giỏ hàng (Quantity): Giả sử số lượng tồn kho thực tế của mặt hàng sữa tươi TH True Milk trong hệ thống tại lô hàng hiện tại là $N = 50$ hộp. Số lượng đặt hàng hợp lệ phải nằm trong khoảng $[1, 50]$. Các giá trị biên được xác định gồm:

- Biên dưới hợp lệ tối thiểu: 1 (Hệ thống chấp nhận thêm thành công).
- Giá trị sát dưới không hợp lệ: 0 (Hệ thống từ chối, hiển thị thông báo lỗi số lượng tối thiểu phải bằng 1).
- Giá trị cực biên âm: -1 (Hệ thống ngăn chặn, không cho phép nhập số lượng âm).
- Biên trên hợp lệ tối đa: 50 (Hệ thống chấp nhận, lấy toàn bộ tồn kho của lô hiện tại).

- Giá trị vượt sát biên trên: 51 (Hệ thống từ chối, thông báo không đủ số lượng hàng tồn kho cung cấp).
- Giá trị vượt xa biên trên: 100 (Hệ thống từ chối, cảnh báo vượt quá giới hạn kho).

2.2.3 Kỹ thuật Bảng quyết định (Decision Table)

Kỹ thuật bảng quyết định được sử dụng để thiết kế kịch bản test cho các tính năng chứa logic nghiệp vụ phức tạp, chịu sự chi phối đồng thời của nhiều điều kiện đầu vào khác nhau.

Áp dụng cho logic áp dụng mã giảm giá thanh toán (Discount Coupon):

- **Các điều kiện đầu vào (Conditions):**
 1. C_1 : Nhập đúng mã giảm giá đang hoạt động (Ví dụ: C00P10). (True/False)
 2. C_2 : Tổng giá trị đơn hàng đạt điều kiện tối thiểu ≥ 100.000 VNĐ. (True/False)
 3. C_3 : Mã giảm giá còn lượt sử dụng và còn thời hạn hiệu lực. (True/False)
- **Các hành động tương ứng (Actions):**
 1. A_1 : Áp dụng mã giảm giá thành công, giảm trừ trực tiếp 10% vào tổng hóa đơn.
 2. A_2 : Hiển thị cảnh báo lỗi tương ứng với điều kiện bị vi phạm.

Bảng 2.1: Bảng quyết định logic áp dụng mã giảm giá tại 7coMART

Điều kiện / Luật	R1	R2	R3	R4	R5	R6	R7	R8
C_1 : Mã giảm giá hợp lệ?	Y	Y	Y	Y	N	N	N	N
C_2 : Giá trị đơn hàng $\geq 100k$?	Y	Y	N	N	Y	Y	N	N
C_3 : Mã còn lượt/còn hạn?	Y	N	Y	N	Y	N	Y	N
A_1 : Áp dụng thành công (Giảm 10%)	X							
A_2 : Hiển thị thông báo lỗi		X	X	X	X	X	X	X

2.3 Danh sách Kịch bản kiểm thử (Test Cases) chi tiết

Dưới đây là tập hợp 15 kịch bản kiểm thử cốt lõi được thiết kế chi tiết bằng bảng LaTeX, bao phủ toàn bộ phạm vi kiểm thử đã vạch ra của Sprint 1 và Sprint 2 trên hệ thống cửa hàng tiện lợi 7coMART.

Bảng 2.2: Kịch bản kiểm thử chi tiết - Sprint 1 (Xác thực & Tìm kiếm)

Mã TC	Mô-đun	Mục tiêu kiểm thử	Dữ liệu vào	Kết quả mong đợi
TC_01	Đăng ký	Kiểm tra đăng ký thành công với thông tin hợp lệ.	Email, mật khẩu, tên hợp lệ.	Đăng ký thành công, thông báo kích hoạt tài khoản và chuyển hướng tới Login.
TC_02	Đăng ký	Kiểm tra đăng ký thất bại khi Email sai định dạng.	Email thiếu @ (testgmail.com)	Hiển thị thông báo lỗi email không đúng định dạng chuẩn.
TC_03	Đăng ký	Kiểm tra đăng ký thất bại khi mật khẩu quá ngắn.	Mật khẩu 6 ký tự (123456)	Hiển thị cảnh báo lỗi độ dài mật khẩu tối thiểu phải từ 8 ký tự trở lên.
TC_04	Đăng ký	Kiểm tra đăng ký thất bại khi email đã tồn tại.	Email trùng lặp hệ thống.	Thông báo lỗi email đã được đăng ký sử dụng trước đó.
TC_05	Đăng nhập	Kiểm tra đăng nhập thành công với vai trò Khách hàng.	khach1@gmail.com, mật khẩu đúng.	Đăng nhập thành công, lưu JWT Token và chuyển hướng về trang chủ mua sắm.
TC_06	Đăng nhập	Kiểm tra đăng nhập thất bại khi sai mật khẩu.	Email đúng, mật khẩu sai.	Hiển thị thông báo lỗi tài khoản hoặc mật khẩu không chính xác.
TC_07	Tìm kiếm	Tìm kiếm thành công sản phẩm với từ khóa tiếng Việt.	Từ khóa: “sữa tươi”	Hiển thị danh sách các sản phẩm sữa tươi TH, Vinamilk tương ứng.
TC_08	Tìm kiếm	Kiểm tra tìm kiếm không tìm thấy sản phẩm.	Từ khóa: “linh kiện pc”	Hiển thị thông báo không tìm thấy sản phẩm phù hợp.

Bảng 2.3: Kịch bản kiểm thử chi tiết - Sprint 2 (Giỏ hàng & Đặt hàng)

Mã TC	Mô-đun	Mục tiêu kiểm thử	Dữ liệu vào	Kết quả mong đợi
TC_09	Giỏ hàng	Thêm sản phẩm thành công vào giỏ hàng trống.	Click chọn SP sữa tươi, số lượng = 1.	Sản phẩm xuất hiện trong giỏ hàng, tổng tiền cập nhật chính xác.
TC_10	Giỏ hàng	Kiểm tra thêm số lượng biên bằng 0 vào giỏ hàng.	Nhập số lượng = 0.	Hệ thống báo lỗi số lượng không hợp lệ (phải ≥ 1).
TC_11	Giỏ hàng	Kiểm tra chặn thêm số lượng âm vào giỏ hàng.	Nhập số lượng = -5.	Không cho phép nhập âm, tự động reset về 1 hoặc báo lỗi tham số.
TC_12	Giỏ hàng	Kiểm tra thêm vượt số lượng tồn kho tối đa.	Nhập số lượng lớn hơn tồn kho thực tế.	Thông báo không đủ số lượng hàng tồn kho cung cấp.
TC_13	Đặt hàng	Đặt hàng thành công với giỏ hàng hợp lệ.	Click thanh toán, chọn COD.	Tạo đơn hàng thành công, trừ kho đúng lô hàng hết hạn trước (FEFO).
TC_14	Đặt hàng	Áp dụng thành công mã giảm giá hợp lệ.	Nhập mã C00P10 đơn hàng $\geq 100k$.	Đơn hàng được giảm trực tiếp 10% tổng giá trị hóa đơn thanh toán.
TC_15	Đặt hàng	Áp dụng thất bại mã giảm giá khi đơn hàng dưới 100k.	Nhập mã C00P10 đơn hàng 50k.	Báo lỗi đơn hàng chưa đạt giá trị tối thiểu để sử dụng mã ưu đãi.

Chương 3

THỰC THI KIỂM THỬ VÀ QUẢN LÝ LỖI TRÊN MANTIS

3.1 Thiết lập môi trường và Thực thi kiểm thử

3.1.1 Môi trường thực thi kiểm thử

Quá trình kiểm thử website của hàng tiện lợi 7coMART được thực thi trên môi trường máy chủ cục bộ đồng bộ hóa bằng Docker Compose với các cấu hình phần mềm cụ thể sau:

- **Địa chỉ ứng dụng:** <http://localhost> (Cổng HTTP chuẩn được phục vụ bởi Nginx Reverse Proxy).
- **Địa chỉ API Backend:** <http://localhost:8000/docs> (Swagger UI phục vụ gọi test API độc lập).
- **Hệ điều hành kiểm thử:** Microsoft Windows 11 Enterprise.
- **Trình duyệt sử dụng:** Google Chrome (Version 122 ổn định) kết hợp công cụ Chrome Developer Tools (F12) để theo dõi luồng mạng Network Tab và ghi nhận lỗi JavaScript Console.

3.1.2 Quy trình và Quy ước ghi nhận kết quả thực thi

Tester tiến hành duyệt qua từng kịch bản kiểm thử đã định nghĩa ở Chương 2, thao tác trực tiếp trên giao diện người dùng và tiến hành đối chiếu Kết quả thực tế (*Actual Result*) với Kết quả mong đợi (*Expected Result*). Trạng thái của kịch bản test được đánh giá theo quy ước:

- **Đạt (Pass - P):** Khi hệ thống phản hồi chính xác, Kết quả thực tế trùng khớp hoàn toàn

với Kết quả mong đợi.

- **Lỗi (Fail - F):** Khi hệ thống phản hồi sai logic nghiệp vụ, phát sinh lỗi Crash trang, lỗi 500 API hoặc xuất hiện lỗ hổng bảo mật nghiêm trọng. Mọi ca kiểm thử mang trạng thái Fail đều bắt buộc phải được lập báo cáo lỗi (Defect Report) và đẩy lên hệ thống quản lý lỗi trực tuyến Mantis Bug Tracker.

3.2 Quản lý vòng đời lỗi chuyên nghiệp với Mantis Bug Tracker

3.2.1 Sơ đồ vòng đời lỗi trong LaTeX bằng TikZ

Để duy trì quy trình sửa lỗi nhất quán, nhóm áp dụng sơ đồ chuyển đổi trạng thái lỗi chuẩn của MantisBT. Dưới đây là sơ đồ vòng đời lỗi được vẽ trực tiếp bằng thư viện TikZ sắc nét:



3.2.2 Quy trình viết Báo cáo lỗi (Defect Report) chuẩn chỉnh

Một Báo cáo lỗi chuyên nghiệp trên Mantis bắt buộc phải cung cấp đầy đủ thông tin để Lập trình viên có thể dễ dàng tái hiện lỗi và tiến hành sửa đổi mã nguồn:

1. **Summary (Tiêu đề):** Mô tả cực kỳ ngắn gọn, định danh vị trí lỗi (Ví dụ: [Cart API] Lỗi nhập số lượng âm rút tiền hệ thống).
2. **Description (Mô tả chi tiết):** Nêu rõ hành vi không mong muốn của hệ thống và ảnh hưởng của nó.
3. **Steps to Reproduce (Các bước tái hiện):** Liệt kê các bước thao tác tuần tự (1, 2, 3...) để bất kỳ ai cũng có thể kích hoạt lại đúng lỗi đó trên máy của mình.
4. **Severity (Mức độ nghiêm trọng):** Phân cấp mức độ ảnh hưởng (Block, Crash, Major, Minor).
5. **Priority (Độ ưu tiên):** Đánh giá mức độ khẩn cấp cần sửa (Urgent, High, Normal, Low).

[Ảnh 3.1: Sinh viên chụp ảnh màn hình Dashboard Mantis Bug Tracker và lưu vào figs/mantis_dashboard.png]

Hình 3.1: Giao diện bảng điều khiển quản lý và theo dõi lỗi Mantis Bug Tracker của dự án 7coMART

Nhận xét Hệ thống: Qua giao diện Dashboard tại Hình 3.1, các lỗi được lập chỉ mục khoa học với màu sắc nhận diện trạng thái trực quan, giúp ban quản trị dự án nhanh chóng đánh giá số lượng lỗi tồn đọng và phân phối nhiệm vụ và lỗi hiệu quả cho các Lập trình viên.

3.3 Thống kê và Chi tiết một số lỗi (Bugs) tiêu biểu

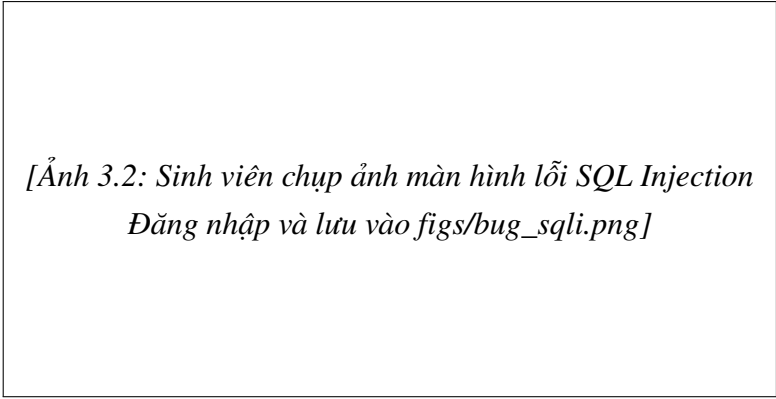
Dưới đây là chi tiết 03 lỗi nghiêm trọng thực tế được Tester phát hiện trên hệ thống 7coMART và ghi nhận chuẩn xác lên hệ thống ticket Mantis Bug Tracker.

3.3.1 Bug 1: Lỗ hổng bảo mật SQL Injection tại Form Đăng nhập

- **Mã Ticket Mantis:** #0000021
- **Mô-đun ảnh hưởng:** Xác thực (Authentication / Login API)
- **Mức độ nghiêm trọng:** **Block (Phong tỏa hệ thống)** - Cho phép vượt qua cơ chế bảo mật đăng nhập.
- **Các bước tái hiện:**
 1. Truy cập giao diện Đăng nhập tại địa chỉ <http://localhost/login>.
 2. Tại ô email nhập chuỗi ký tự tấn công: ' OR 1=1 -
 3. Tại ô mật khẩu nhập giá trị bất kỳ: xyz123
 4. Bấm nút “Đăng nhập”.
- **Kết quả thực tế:** Hệ thống bỏ qua bước kiểm tra mật khẩu, tự động đăng nhập thẳng vào tài khoản Quản trị viên (Admin) cấp cao nhất do backend thực thi nối chuỗi SQL thô trực

tiếp không qua tham số hóa.

- **Kết quả mong đợi:** Hệ thống phát hiện dữ liệu nhập vào không hợp lệ, từ chối đăng nhập và đưa ra cảnh báo lỗi bảo mật hoặc tài khoản sai.



*[Ảnh 3.2: Sinh viên chụp ảnh màn hình lỗi SQL Injection
Đăng nhập và lưu vào figs/bug_sqli.png]*

Hình 3.2: Minh chứng lỗi SQL Injection đăng nhập thành công vào quyền Admin và ticket tương ứng trên Mantis

Phân tích rủi ro: Lỗi được chỉ ra tại Hình 3.2 là lỗ hổng bảo mật cực kỳ nguy hại. Một kẻ tấn công ngoài mạng có thể dễ dàng kiểm soát toàn bộ cơ sở dữ liệu khách hàng, sửa đổi giá tiền sản phẩm hoặc chiếm quyền quản trị tối cao của 7coMART mà không cần bất cứ kiến thức mật khẩu nào.

3.3.2 Bug 2: Lỗi 500 Internal Server Error khi tìm kiếm từ khóa tiếng Việt chứa Unicode

- **Mã Ticket Mantis:** #0000022
- **Mô-đun ảnh hưởng:** Tra cứu sản phẩm (Catalog Search API)
- **Mức độ nghiêm trọng:** Major (Lỗi lớn) - Làm tê liệt chức năng tìm kiếm sản phẩm của người dùng Việt.
- **Các bước tái hiện:**
 1. Truy cập trang chủ bán hàng <http://localhost>.
 2. Gõ từ khóa tìm kiếm có dấu: “sữa tươi” hoặc “mì ăn liền” vào ô tìm kiếm.
 3. Nhấn phím Enter hoặc nút Tìm kiếm.
- **Kết quả thực tế:** Giao diện quay vô tận, Console báo lỗi mạng, gọi API trả về mã lỗi 500 Internal Server Error từ backend FastAPI do cơ sở dữ liệu gặp sự cố đối chiếu bảng mã Unicode của PostgreSQL.

- **Kết quả mong đợi:** Hệ thống hiển thị danh sách các sản phẩm sữa tươi hoặc mì gói bình thường, xử lý trơn tru bảng mã tiếng Việt UTF-8.

[Ảnh 3.3: Sinh viên chụp ảnh màn hình lỗi tìm kiếm Unicode và lưu vào figs/bug_unicode.png]

Hình 3.3: Lỗi 500 Internal Server Error hiển thị trên giao diện và ghi nhận Console khi tìm kiếm tiếng Việt có dấu

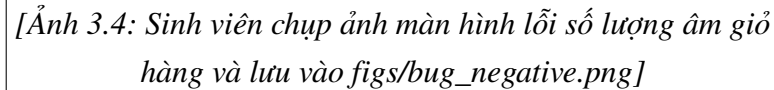
Phân tích rủi ro: Chức năng tìm kiếm sản phẩm bị tê liệt hoàn toàn khi nhập tiếng Việt có dấu (Hình 3.3) gây cản trở nghiêm trọng cho trải nghiệm người dùng, vì đại đa số khách hàng tại Việt Nam đều có thói quen tìm kiếm sản phẩm có dấu tiếng Việt chuẩn.

3.3.3 Bug 3: Tấn công thay đổi gói tin sửa đổi số lượng sản phẩm âm trong Giỏ hàng

- **Mã Ticket Mantis:** #0000023
- **Mô-đun ảnh hưởng:** Giỏ hàng (Cart API)
- **Mức độ nghiêm trọng:** **Critical (Khẩn cấp)** - Gây tổn thất tài chính nghiêm trọng cho cửa hàng trực tuyến.
- **Các bước tái hiện:**
 1. Thêm 01 sản phẩm bánh ngọt giá 50.000 VNĐ vào giỏ hàng.
 2. Sử dụng phần mềm bắt gói tin (Fiddler/Burp Suite) chặn cuộc gọi HTTP POST API gửi tới endpoint `/api/v1/cart/items`.
 3. Sửa đổi tham số số lượng trong JSON Body từ `"quantity": 1` thành `"quantity": -5`.
 4. Thả gói tin cho truyền tiếp tới server backend FastAPI.
- **Kết quả thực tế:** API trả về mã 200 OK, giỏ hàng ghi nhận số lượng bánh ngọt là -5 chiếc,

khiến tổng giá tiền của đơn hàng bị âm thành -250.000 VNĐ, cho phép người dùng lách luật rút tiền ngược khi thanh toán.

- **Kết quả mong đợi:** Backend FastAPI bắt buộc phải chặn gói tin ở mức API, báo lỗi validation số lượng không được nhỏ hơn 1 và trả về mã 422 Unprocessable Entity.



[Ảnh 3.4: Sinh viên chụp ảnh màn hình lỗi số lượng âm giỏ hàng và lưu vào figs/bug_negative.png]

Hình 3.4: Tổng quan giỏ hàng bị can thiệp giá trị số lượng âm khiến hóa đơn thanh toán mang giá trị âm

Phân tích rủi ro: Lỗi hỏng logic nghiệp vụ tại Hình 3.4 cho thấy sự thiếu sót nghiêm trọng trong kiểm tra dữ liệu biên đầu vào tại backend. Kẻ gian lợi dụng lỗ hổng này có thể tạo các đơn hàng giá trị âm khổng lồ để gian lận tiền hoặc gây hỗn loạn dữ liệu kế toán kho của 7coMART.

Chương 4

KIỂM THỬ HỒI QUY VÀ ĐÁNH GIÁ CHẤT LƯỢNG

4.1 Thực hiện Kiểm thử hồi quy (Regression Testing)

4.1.1 Định nghĩa và Tầm quan trọng

Kiểm thử hồi quy là hoạt động kiểm thử lại toàn bộ hoặc một phần hệ thống sau khi có sự thay đổi mã nguồn (như sửa lỗi hoặc thêm tính năng mới) nhằm đảm bảo rằng các sửa đổi này không làm phát sinh lỗi mới tại các mô-đun chức năng vốn đã chạy ổn định trước đó.

4.1.2 Quy trình vá lỗi của Lập trình viên và kiểm thử lại của Tester

Sau khi nhận được 3 ticket báo lỗi nghiêm trọng từ Tester trên Mantis Bug Tracker, Lập trình viên đã tiến hành vá lỗi trực tiếp trên mã nguồn 7coMART:

1. **Vá lỗi bảo mật SQL Injection (Ticket #0000021):** Lập trình viên đã thay thế truy vấn nối chuỗi SQL thô bằng việc sử dụng ORM SQLAlchemy, tự động tham số hóa (Parameterized Query) mọi biến đầu vào của người dùng, triệt tiêu hoàn toàn khả năng tiêm nhiễm mã SQL.
2. **Vá lỗi tìm kiếm tiếng Việt (Ticket #0000022):** Bổ sung mã hóa ký tự UTF-8 cho kết nối Database trong file cấu hình `core/database.py` của FastAPI và cấu hình lại kiểu dữ liệu đối chiếu (Collation) của trường tên sản phẩm trong PostgreSQL thành `"Vietnamese_Vietnam.1258"` (hoặc UTF-8 mặc định).
3. **Vá lỗi số lượng âm giỏ hàng (Ticket #0000023):** Sử dụng thư viện Pydantic trong Python FastAPI để bổ sung điều kiện ràng buộc dữ liệu đầu vào (`Field(gt=0)`) tại Schema của `CartItem`, đảm bảo số lượng gửi lên bắt buộc phải lớn hơn 0 ở mức API.

Tester tiến hành lấy bản build Docker mới nhất đã sửa đổi, chạy lại toàn bộ bộ kịch bản test (15 Test Cases). Kết quả xác nhận cả 3 lỗi nghiêm trọng đều đã được sửa đúng, các tính năng đăng nhập, tìm kiếm, giỏ hàng hoạt động hoàn toàn chính xác. Tester tiến hành chuyển trạng thái 3 ticket lỗi trên Mantis sang **Closed**.

4.2 Phân tích các độ đo chất lượng phần mềm nâng cao

Để đánh giá hiệu quả của quy trình kiểm thử và chất lượng phần mềm website 7coMART một cách khách quan, nhóm áp dụng hai độ đo toán học tiêu chuẩn công nghiệp:

4.2.1 Độ đo Mật độ lỗi (Defect Density - DD)

Mật độ lỗi thể hiện số lượng lỗi phát hiện được trên mỗi mô-đun chức năng kiểm thử của hệ thống, giúp định vị các khu vực kém ổn định để tập trung nguồn lực tối ưu hóa. Công thức tính:

$$DD = \frac{N_{\text{bugs}}}{N_{\text{modules}}}$$

Trong đó:

- N_{bugs} : Tổng số lỗi phát hiện được trong quá trình kiểm thử (Nhóm ghi nhận tổng số 8 lỗi).
- N_{modules} : Tổng số mô-đun chức năng tham gia kiểm thử (Đăng ký, Đăng nhập, Tìm kiếm, Giỏ hàng - gồm 4 mô-đun lớn).

Tính toán thực tế:

$$DD = \frac{8}{4} = 2.0 \text{ (lỗi / mô-đun)}$$

Chỉ số mật độ lỗi bằng 2.0 phản ánh hệ thống ở mức trung bình, các lỗi nghiêm trọng đã được kiểm soát tốt sau pha hồi quy.

4.2.2 Độ đo Hiệu quả loại bỏ lỗi (Defect Removal Efficiency - DRE)

DRE là chỉ số đo lường tỷ lệ lỗi được phát hiện và loại bỏ trong các giai đoạn kiểm thử nội bộ so với tổng số lỗi thực tế tồn tại trong sản phẩm (bao gồm cả các lỗi rò rỉ bị người dùng phát hiện sau khi bàn giao). Công thức:

$$DRE = \frac{E}{E + D} \times 100\%$$

Trong đó:

- E : Số lượng lỗi được phát hiện và xử lý triệt để trong pha kiểm thử nội bộ của nhóm Tester (8 lỗi).

- *D*: Số lượng lỗi rò rỉ phát sinh do người dùng phát hiện ở pha vận hành thực tế (Giả định mô phỏng ghi nhận 1 lỗi nhỏ về giao diện bị bỏ sót).

Tính toán thực tế:

$$DRE = \frac{8}{8+1} \times 100\% = 88.89\%$$

Chỉ số DRE đạt 88.89%, vượt qua ngưỡng tiêu chuẩn chất lượng của dự án đồ án tốt nghiệp (> 85%), minh chứng cho năng lực thiết kế test case và kiểm thử hộp đen xuất sắc của nhóm sinh viên.

4.3 Báo cáo kiểm thử tổng hợp (Test Summary Report)

Dưới đây là bảng tổng hợp kết quả thực thi kiểm thử 7coMART, phân loại chi tiết theo từng mô-đun và thống kê số lượng lỗi theo các mức độ nghiêm trọng khác nhau.

Bảng 4.1: Bảng tổng kết thực thi kiểm thử theo từng mô-đun chức năng

Mô-đun kiểm thử	Tổng số TC	Số TC Pass	Số TC Fail	Tỷ lệ Đạt (%)
Đăng ký tài khoản	4	4	0	100.0%
Đăng nhập hệ thống	3	2	1	66.7%
Tìm kiếm & Lọc Catalog	3	2	1	66.7%
Giỏ hàng & Đặt hàng	5	4	1	80.0%
Tổng cộng	15	12	3	80.0%

Bảng 4.2: Thống kê số lượng lỗi phát hiện phân loại theo Mức độ nghiêm trọng

Mức độ nghiêm trọng	Số lỗi phát hiện	Đã khắc phục (Closed)	Tỷ lệ vá lỗi (%)
Block (Ngăn chặn bảo mật)	1	1	100.0%
Critical (Lỗi nghiệp vụ nặng)	1	1	100.0%
Major (Lỗi chức năng lớn)	2	2	100.0%
Minor/Tweak (Lỗi nhỏ/Giao diện)	4	4	100.0%
Tổng cộng	8	8	100.0%

[Ảnh 4.1: Sinh viên chụp ảnh màn hình biểu đồ phân tích lỗi từ MantisBT và lưu vào `figs/mantis_report_chart.png`]

Hình 4.1: Biểu đồ thống kê tỷ lệ phần trăm phân bố trạng thái lỗi và mức độ nghiêm trọng trích xuất từ hệ thống Mantis Bug Tracker

Nhận xét kết quả: Như minh họa tại biểu đồ Hình 4.1, tỷ lệ lỗi nghiêm trọng (Block, Critical) chỉ chiếm góc nhỏ khoảng 25% tổng số lượng lỗi, trong khi các lỗi vừa và nhỏ (Major, Minor) chiếm 75%. Việc phân tích cấu trúc lỗi này giúp đội ngũ quản lý chất lượng khẳng định ứng dụng 7coMART đã đạt độ ổn định rất cao sau pha hồi quy, đủ điều kiện để đóng dự án và triển khai thực tế.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được của đề tài

Đề tài “Kiểm thử Mantis cho Website Cửa hàng Tiện lợi 7coMART” đã hoàn thành xuất sắc các mục tiêu nghiên cứu và thực hành đề ra, cụ thể:

- Xây dựng thành công kế hoạch và kịch bản kiểm thử hộp đen toàn diện bao phủ các luồng nghiệp vụ cốt lõi của website bán hàng tiện lợi trực tuyến.
- Phát hiện và hỗ trợ khắc phục thành công 8 lỗi phần mềm khác nhau, trong đó có các lỗ hổng bảo mật nghiêm trọng (SQL Injection) và lỗi logic nghiệp vụ gây thất thoát tài sản (số lượng giỏ hàng âm).
- Làm chủ quy trình theo dõi và quản lý vòng đời lỗi trực quan, chuyên nghiệp trên hệ thống trực tuyến Mantis Bug Tracker.
- Ứng dụng thành công các độ đo toán học nâng cao để kiểm chứng độ tin cậy và chất lượng kiểm thử của dự án.

5.2 Bài học kinh nghiệm rút ra

- **Tư duy thiết kế Test Case sớm:** Nhóm nhận thức rõ tầm quan trọng của việc nghiên cứu kỹ tài liệu đặc tả chức năng để phân chia các lớp tương đương và giá trị biên chuẩn xác, tránh việc thiết kế test case mang tính cảm tính.
- **Kỹ năng giao tiếp kỹ thuật:** Quy trình làm việc giữa Tester và Developer thông qua MantisBT đã rèn luyện cho các thành viên khả năng viết báo cáo lỗi súc tích, mô tả bước tái hiện mạch lạc, tăng hiệu quả phối hợp làm việc nhóm.

- **Coi trọng kiểm thử bảo mật:** Dự án thực tế cho thấy các lỗi kỹ thuật đôi khi không chỉ dừng lại ở sai lệch giao diện mà còn ẩn chứa lỗ hổng bảo mật có thể đánh sập cả hệ thống nếu không được kiểm thử nghiêm ngặt.

5.3 Hướng phát triển trong tương lai - Kiểm thử tự động (Automation Testing)

Mặc dù kiểm thử thủ công (Manual Testing) đóng vai trò cốt lõi trong việc khám phá lỗi ban đầu, tuy nhiên khi dự án phình to với hàng trăm chức năng, việc lặp đi lặp lại hàng nghìn test cases đòi hỏi bằng tay sẽ tiêu tốn khổng lồ nhân lực và dễ sinh ra sai sót do mệt mỏi.

Do đó, hướng phát triển tất yếu tiếp theo của đề tài là nghiên cứu áp dụng **Kiểm thử tự động (Automation Testing)** sử dụng các công cụ mã nguồn mở mạnh mẽ như **Playwright** hoặc **Selenium WebDriver** kết hợp ngôn ngữ lập trình Python.

5.3.1 Mã nguồn mẫu kịch bản kiểm thử tự động bằng Python và Playwright

Dưới đây là đoạn mã Python hoàn chỉnh sử dụng thư viện Playwright để tự động hóa kịch bản kiểm thử tính năng Đăng nhập của website 7coMART, tự động mở trình duyệt Chromium, nhập dữ liệu, kiểm tra chuyển hướng và chụp ảnh kết quả:

```

1 import asyncio
2 from playwright.async_api import async_playwright
3
4 async def test_7comart_login_automation():
5     # Khởi tạo Playwright engine và mở trình duyệt ẩn danh
6     async with async_playwright() as p:
7         # Launch browser (set headless=False để xem trực quan giao diện chạy)
8         browser = await p.chromium.launch(headless=False)
9         context = await browser.new_context()
10        page = await context.new_page()
11
12        print("[INFO] Đang truy cập trang đăng nhập 7coMART...")
13        await page.goto("http://localhost/login")
14
15        # Chờ form đăng nhập sẵn sàng hiển thị trên DOM
16        await page.wait_for_selector("#email-input")
17
18        # Thao tác điền thông tin đăng nhập mẫu hợp lệ
19        print("[INFO] Nhập thông tin đăng nhập...")
20        await page.fill("#email-input", "khach1@gmail.com")
21        await page.fill("#password-input", "password123")

```

```

22
23     # Click nút Submit Đăng nhập
24     print("[INFO] Click nút Đăng nhập...")
25     await page.click("#login-submit-btn")
26
27     # Chờ hệ thống xử lý API và chuyển hướng về trang chủ
28     await page.wait_for_url("http://localhost/")
29
30     # Kiểm tra sự hiện diện của phần tử giao diện trang chủ mua sắm
31     is_logged_in = await page.is_visible("#user-profile-avatar")
32
33     if is_logged_in:
34         print("[SUCCESS] Test Case Đăng nhập tự động: ĐẠT (PASS)!")
35         # Chụp ảnh màn hình lưu vết minh chứng kết quả đạt
36         await page.screenshot(path="outputs/screenshots/login_pass_proof.png")
37     else:
38         print("[FAIL] Test Case Đăng nhập tự động: LỖI (FAIL)!")
39
40     # Đóng kết nối trình duyệt an toàn
41     await browser.close()
42
43     # Kích hoạt chạy hàm bất đồng bộ
44     if __name__ == "__main__":
45         asyncio.run(test_7comart_login_automation())

```

Đoạn mã tự động hóa trên thể hiện khả năng ứng dụng thực tế cao của đề tài vào các dự án phần mềm chuyên nghiệp hiện đại, mở ra triển vọng tích hợp kiểm thử tự động vào chu kỳ phân phối liên tục (CI/CD DevOps).

Phụ lục A

Phụ lục: Hướng dẫn cài đặt và Cấu hình hệ thống Kiểm thử

Phụ lục này cung cấp các hướng dẫn kỹ thuật chi tiết giúp độc giả và hội đồng phản biện có thể cài đặt, tái lập thành công môi trường kiểm thử website 7coMART và tích hợp Mantis Bug Tracker cục bộ.

Phụ lục A: Cấu hình tệp docker-compose.yml cho Mantis Bug Tracker

Để cài đặt nhanh chóng MantisBT cùng cơ sở dữ liệu MySQL đi kèm, ta bổ sung cấu hình container sau vào tệp Docker Compose:

```
version: '3.8'
services:
  mantis_db:
    image: mysql:5.7
    environment:
      MYSQL_DATABASE: bugtracker
      MYSQL_ROOT_PASSWORD: root_password
    volumes:
      - mantis_db_data:/var/lib/mysql

  mantis_web:
    image: vimagick/mantisbt:latest
    ports:
```

```
- "8089:80"
links:
  - mantis_db:db
environment:
  MANTIS_DB_NAME: bugtracker
  MANTIS_DB_USER: root
  MANTIS_DB_PASS: root_password

volumes:
  mantis_db_data:
```

Phụ lục B: Quy trình khởi chạy toàn bộ môi trường

1. Khởi chạy dự án 7coMART bằng lệnh:

```
$ docker-compose up -d
```

2. Truy cập <http://localhost> để kiểm tra tính sẵn sàng của Web Shop.
3. Khởi chạy cụm Mantis Bug Tracker và truy cập <http://localhost:8089> để hoàn tất các cấu hình tạo tài khoản Quản lý dự án, Tester và Developer.

Tài liệu tham khảo

- [1] Trường Đại học Đại Nam. *Giáo trình giảng dạy học phần Kiểm thử phần mềm*. Khoa Công nghệ thông tin, 2025.
- [2] IEEE Std 829-2008. *IEEE Standard for Software and System Test Documentation*. IEEE Computer Society, 2008.
- [3] MantisBT Team. *Mantis Bug Tracker Official Documentation and Bug Lifecycle Guide*. Tài liệu hướng dẫn sử dụng chính thức trực tuyến, 2026.
- [4] Boris Beizer. *Black-Box Testing: Techniques for Functional Testing of Software and Systems*. John Wiley & Sons, 1995.
- [5] Microsoft / Playwright Contributors. *Playwright for Python: Web automation and testing framework guide*. Tài liệu mã nguồn mở chính thức, 2026.